Top 5 Caching Patterns

#11: Read Now - How to Update the Cache (3 minutes)



NEO KIM
OCT 05, 2023

95 10 9

Get my system design playbook for FREE on newsletter signup:

Type your email... Subscribe

This post outlines the most common caching patterns. If you want to learn more, s the bottom and find the references.

- *Share this post & I'll send you some rewards for the referrals.*

A cache is a temporary storage that provides quick access to frequently used dat it's usually stored in memory for low latency. But available memory is limited. So important to update the cache the right way.

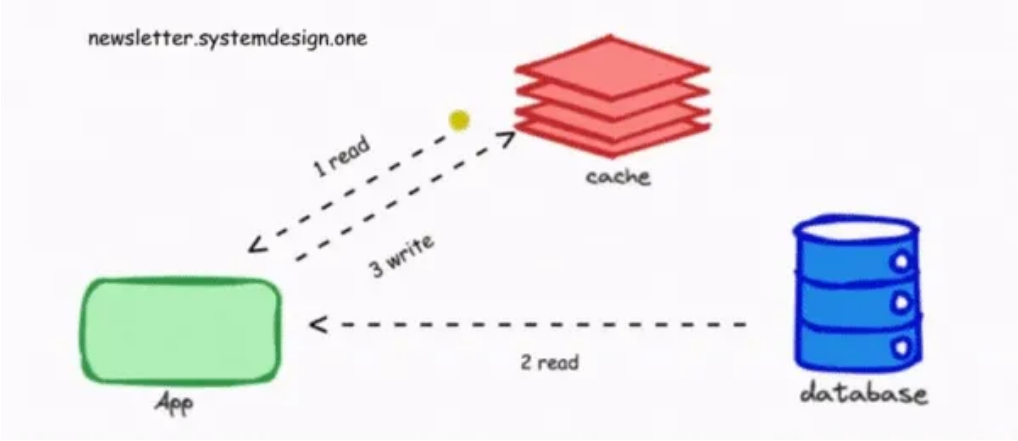
I will teach you the popular caching patterns in this post.

A *cache hit* occurs when queried data is available in the cache. And a *cache miss* when queried data isn't available in the cache.

Caching Patterns

The 5 popular caching patterns are:

Manage Reject Accept



Cache Aside Pattern

Here is the *workflow*:

1. Read data from the cache
2. Read data from the database on a cache miss
3. Write data to the cache
4. And return data

It's also called *lazy loading*. Because only the queried data gets cached.

Also the cache doesn't interact with the database. But the app does.

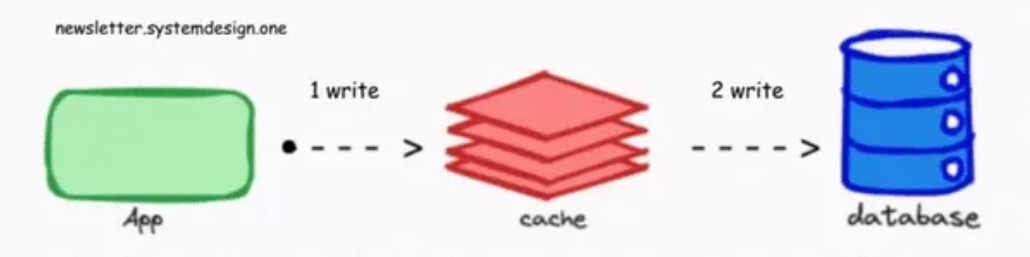
Advantages

- Easy implementation
- Finer control over cache population. Because only frequently accessed data i cached

Disadvantages

- Degraded latency. Because there are many trips
- Potential data inconsistency

2. Write Through



Write Through Pattern

Here is the *workflow*:

1. Write data to the cache
2. And the cache writes data *synchronously* to the database

The app doesn't interact with the database. But the cache does.

Advantages

- Data consistency
- Low read latency. Because the cache contains fresh data

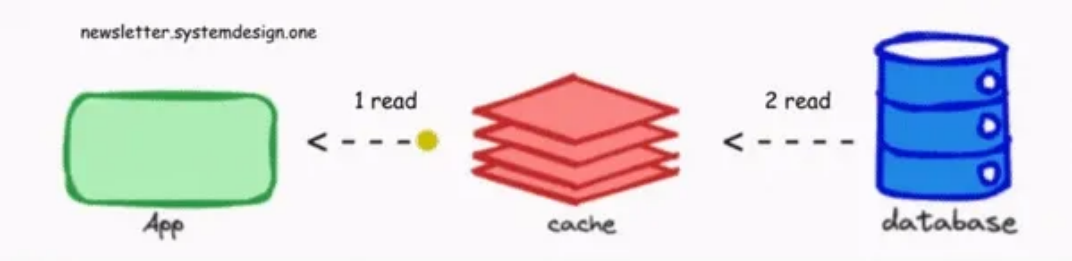
Disadvantages

- High write latency. Because both cache and database need to be updated
- Most data in the cache is never read

Use Cases

- A low number of writes expected
- Data freshness is important

3. Read Through



Read Through Pattern

Here is the *workflow*:

1. Read data from the cache
2. Read data from the database on a cache miss
3. Write data to the cache
4. And return data

The app doesn't interact with the database. But the cache does.

And this is what makes it different from the cache aside pattern.

Advantages

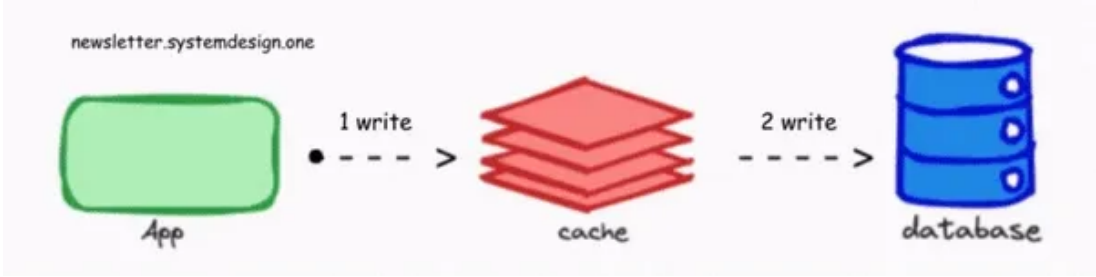
- Low read latency for frequently accessed data
- Improved read scalability

Disadvantages

- Potential data inconsistency. Because the cache might contain stale data
- High latency on a cache miss

Use Cases

- Read-heavy workloads
- A high cache miss rate is acceptable



Write Back Pattern

Here is the *workflow*:

1. Write data to the cache
2. And the cache writes data *asynchronously* to the database

It's also called the *write-behind* pattern.

Advantages

- Low write latency
- Improved performance. Because writes to the database get batched

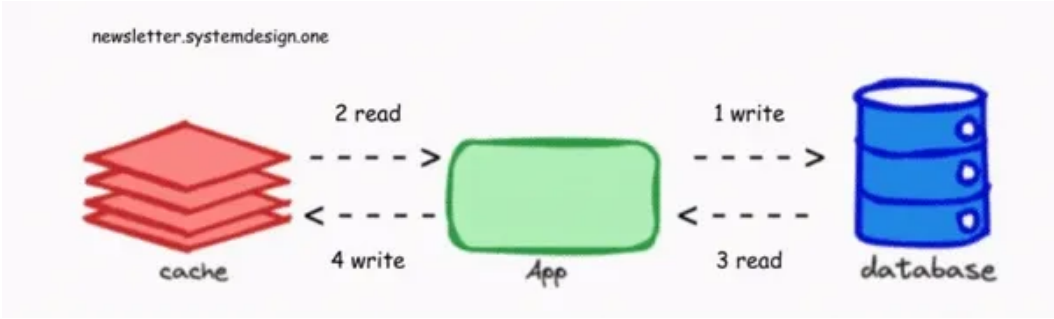
Disadvantages

- Increased risk of data loss. Because the cache might fail before writing to the database
- Complex implementation

Use Cases

- Write-heavy workloads
- Data durability is not important

5. Write Around



Write Around Pattern

Here is the *workflow*:

1. Write data to the database
2. Read data from the cache
3. Read data from the database on a cache miss
4. And write data to the cache

Advantages

- Reduced risk of data loss
- Reduced cache pollution. Because cache stores only frequently accessed dat

Disadvantages

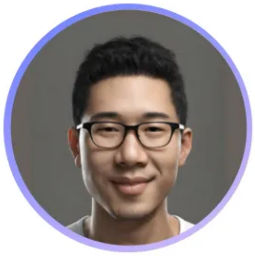
- High read latency
- High cache miss rate

Use Cases

- No data update expected
- A low number of reads expected

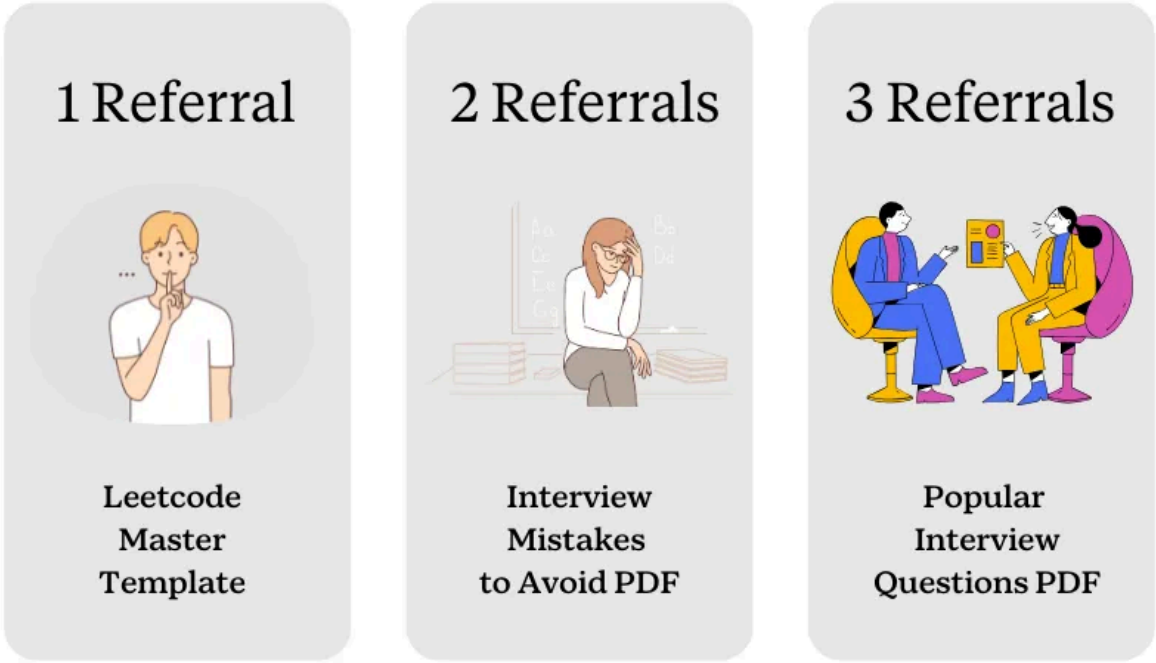
Redis and Memcached are popular cache implementations. And a combination o

Type your email... Subscribe



Follow me on LinkedIn | YouTube | Threads | Twitter | Instagram

Thank you for supporting this newsletter. Consider sharing this post with your fri and get rewards. Y'all are the best.





This Is How Stripe Does Rate Limiting to Build Scalable APIs

NK • 28. SEPTEMBER 2023

Read full story →

And don't forget to hit the Like button if you enjoyed this post ❤️

References

- https://www.prisma.io/dataguide/managing-databases/introduction-databas caching
- https://www.slideshare.net/tmatyashovsky/from-cache-to-in-memory-data-g introduction-to-hazelcast
- https://www.alachisoft.com/resources/articles/readthru-writethru-writebehind.html



95 Likes · 9 Restacks

← Previous Nex

Discussion about this post

Comments Restacks



Write a comment...

This is a great idea for an article but unfortunately the descriptions are not easy to read. For e
the workflow for Read-through is WORD FOR WORD identical to Cache Aside. The words are
insufficient to explain what's going on, so is the diagram. Readers have to read both the word
diagram to stitch together hat's going on. I would encourage rewriting these to explain in mo

LIKE (2) REPLY

2 replies by Neo Kim and others



Anton Zaides 5. Okt. 2023

Liked by Neo Kim

Interesting! I've worked only with the Cache aside implementation, as it's the easier to mainta

LIKE (1) REPLY

8 more comments...

© 2026 Neo Kim · Publisher Privacy
Substack · Privacy · Terms · Collection notice
Substack is the home for great culture